A Bit-Exact Fixed-Point FPGA Echo State Network Reservoir for Lithium-Ion Battery SOC and SOH Estimation

Akshay Nambiar, Satvik Verma, Tanish Sanghvi, Sanjna Patankar, Kadambari Sharma, and Nilima Warke

{immortalwealth2k4, satvikverma14, tanishjain020205, sanjna.patankar}@gmail.com, {kadambari.sharma, nilima.warke}@ves.ac.in

The authors are with the Department of Automation and Robotics, Vivekanand Education Society's Institute of Technology (VESIT), Mumbai 400074, India.

***Abstract*—A battery management system for lithium-ion cells must report state of charge (SOC) and state of health (SOH), but neither can be measured directly. The standard fixes each fail in a different way: current integration drifts, Kalman observers depend on cell parameters that shift with temperature and battery ageing, and recurrent networks that avoid those models are too heavy to train and run on small embedded hardware. We argue that an echo state network removes the cost that blocks the networks. Its recurrent weights stay fixed and random, only a linear readout is fitted (in closed form), and the whole model can be quantized once. We tested this on an Artix-7 FPGA with a 100-neuron reservoir in Q6.10 fixed point, run through one time-multiplexed multiply-accumulate core and verified bit-exact against a golden model. On NASA PCoE cells B0005, B0006, B0007 and B0018, the pooled SOC R^2 is 0.922 against 0.898 for an extended Kalman filter, using 318 LUTs and two DSP slices. The gain comes from the reservoir, not from the number format.**

***Index Terms*—Battery management system, Echo state network, Extended Kalman filter, Field programmable gate array, Fixed-point arithmetic, Lithium-ion battery, Reservoir computing, State of charge, State of health.**

I. Introduction

Battery state estimation is a problem of inference under drift. SOC and SOH cannot be read from a sensor [1], so every battery management system (BMS) infers them from voltage and current, and each classical route carries an assumption that aging eventually breaks. Coulomb counting accumulates sensor bias. Extended Kalman filter (EKF) observers [2], [3] need cell parameters that must be re-identified, as they change with temperature and capacity fade. Deep recurrent networks such as long short-term memory (LSTM) [4] avoid those parameters but need floating-point arithmetic, backpropagation and memory that a small embedded BMS does not have. Reservoir computing offers a middle path [5]–[7]: an echo state network (ESN) [5], [8] trains only a linear readout in closed form, so its weights can be quantized once and stored on chip. Whether such a reservoir, once quantized and synthesized, can match a classical observer on unseen real cells at small hardware cost has remained open. Without a method that avoids all three, a designer must either accept growing estimation error, or add recalibration, characterization or compute hardware to every pack. The contributions of this paper are:

1) Q6.10 fixed-point ESN datapath with a time-multiplexed MAC core, block-RAM weights, a double-buffered state memory, and a lookup-table tanh;
2) Stage-level bit-exact verification over a complete 64-step real discharge sequence;
3) Leakage-free nested leave-one-cell-out (LOCO) evaluation on NASA PCoE cells against a refitted EKF, with ablations and a ten-seed variability study;
4) Fixes for SOH overfitting under single-cell training that remains one 100-coefficient dot product in hardware, plus post-route resource, timing, and power figures checked module by module.

II. Related Work

Plett's EKF [2] and its sigma-point variant [9] are the reference model-based estimators, and Li *et al.* [3] separate SOC and SOH onto two EKF timescales. Data-driven work [4], [10] rarely reports the memory, latency, or energy needed on embedded hardware. Reservoir computing [5], [6] is reviewed in [7], [11]. Kamarudin *et al.* [12] evaluated a spiking reservoir for SOC on NASA PCoE cells in software only; Gan [13] built a digital ESN on FPGA for generic time-series tasks; Rapuano *et al.* [14] quantize LSTM/GRU accelerators for FPGA/ASIC; and Yun *et al.* [15] implement a fixed-point EKF for SOC on an FPGA. This work joins these threads: a learned reservoir, quantized for hardware, verified bit-exact at RTL, and evaluated on unseen cells against a classical baseline.

III. ESN Formulation and Fixed-Point Design

The reservoir state $\mathbf{x}(t) \in \mathrm{R}^N$ evolves as

$$x(t+1) = tanh\left(W_{in}u(t) + Wx(t) + b\right) \tag{1}$$

where $\mathbf{u}(t) \in \mathrm{R}^4$ holds measured voltage, current, temperature, and load current. The hardware uses $N = 100$ and no leaky integration. $\mathbf{W}$ has 10% density with an 80%/20% excitatory/inhibitory split; after Q6.10 quantization it holds 996 nonzeros in $[-0.295,\ 0.295]$ with spectral radius 0.900, $\mathbf{W}_{\text{in}}$ lies in $[-0.495,\ 0.490]$, and the deployed $\mathbf{b} = \mathbf{0}$. SOC is read out per step and SOH once per cycle from the cycle-mean state $\bar{x}$ projected onto k principal components:

$$\hat{SOC}(t) = w_{SOC}^{\top}x(t) + c_{SOC} \tag{2}$$

$$\hat{SOH} = \beta^{\top}P_k(x^- - \mu) + c_{SOH} = w_{SOH}^{\top}x^- + c'_{SOH} \tag{3}$$

Because the projection is linear, (3) collapses to one 100-coefficient dot product. Coefficients are fitted by ridge regression on the dequantized states of the bit-exact model, so the readouts are trained on exactly what the FPGA computes.

All datapath values are 16-bit signed Q6.10 (range (−32, 32), resolution 2^{-10}). Q12.20 products are summed in a guarded wide accumulator and, after the 104 MACs of one neuron, rescaled with round-half-up,

$$s = (\Sigma + 29) \gg 10 \tag{4}$$

then saturated, biased, and clipped to [−5, 5]. Because tanh is odd, 5121 entries cover [0, 5.0] and the sign is restored after lookup; the table's maximum error against double-precision tanh is 4.88×10^{-4}, half of one least-significant bit.

IV. FPGA Architecture and Verification

Fig. 1 shows the controller handshake that sequences each neuron update: the last recurrent MACs, the done strobe, and the start of the next input-weight phase. Fig. 2 shows the top-level architecture, in which one compute core serves all 100 neurons. The address generator produces the input-weight ($a_$in), recurrent-weight ($a_$W), input($a_u = tNin + k$), and state-read (j) addresses, and two multiplexers feed one MAC core with (W_{in}, u) and then (W, x). For neuron i, the core performs four input and 100 recurrent MACs, rescales by (4), adds the bias, clips, looks up tanh, and writes $x_i(t+1)$: port A writes the new state at neuron index i while port B reads the previous state, and the two state buffers swap roles after each update. The multiply–accumulate maps to two DSP48E1 slices and the tanh table to four RAMB36 blocks.

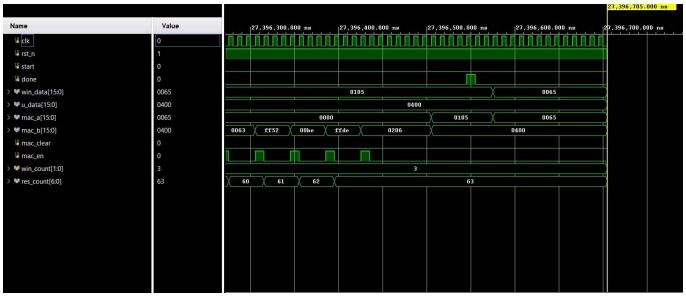


Fig. 1. Controller handshake at the end of Run A: the last recurrent MACs, the done strobe, and the start of the next update's input-weight phase.

Fig. 3 shows Vivado's synthesized top level with every memory and the neuron core retained: the neuron core contains the MAC (two DSP48E1 slices) and the tanh ROM, the six block-RAM cores hold $\mathbf{W}$, $\mathbf{W}_{in}$, the inputs, the biases, and the two state buffers, and $x_i(t+1)$ leaves on x_out[15:0] with its x_valid strobe. Two indices are kept distinct: the neuron index i is the write address, while the recurrent read index $j \in \{0, \ldots, 99\}$ sweeps the previous state; merging them would silently remove the recurrent coupling. Because every neuron needs the complete $\mathbf{x}(t)$, the state memory is a simple dual-port block RAM split into two ping-pong buffers – port A writes one while port B reads the other, and the roles swap after each update – which avoids a Gauss–Seidel-like mixing of $\mathbf{x}(t)$ and $\mathbf{x}(t+1)$. Inputs are stored contiguously and read at $a_u = tN_{in} + k$ for sequences up to 64 steps; all memories total about 175 kbit, dominated by the densely stored recurrent matrix (160 kbit).

The Python golden model reimplements the RTL arithmetic exactly (weights, Q12.20 accumulation, rounding, saturation, tanh table) and was written from the specification, independently of the RTL. For every neuron update, both the golden model and the Vivado behavioral simulation log five values: rounded MAC, bias, biased sum, clipped tanh input, and tanh output – which are joined on (timestep, neuron) and compared word for word, localizing any discrepancy to a stage. *Run A* drives the deployed reservoir with a complete 64-step real discharge (cell B0018, cycle 115; 64 × 100 = 6400 updates); *Run B* uses an earlier weight set with nonzero biases over two updates (200 updates) to exercise the bias path.

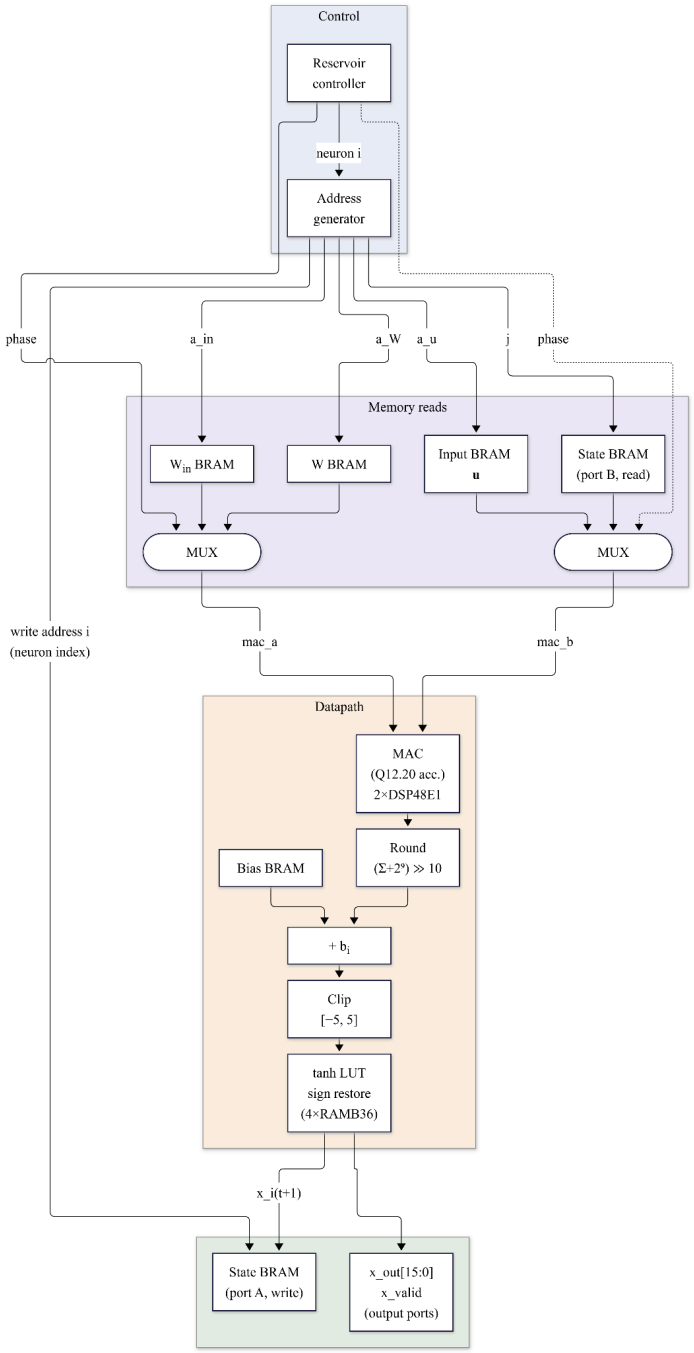


Fig. 2. Top-level FPGA architecture.

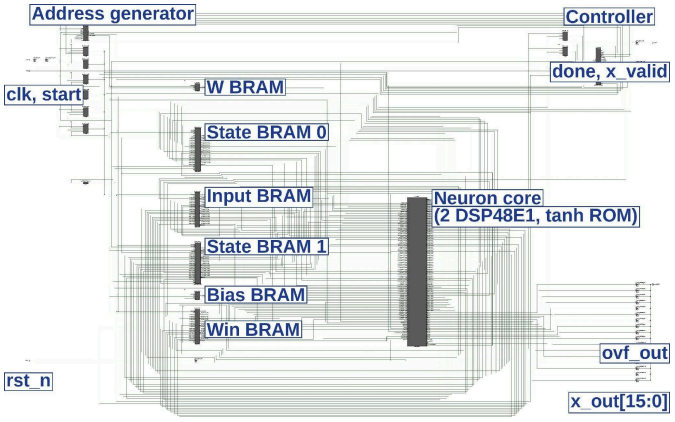


Fig. 3. Vivado-synthesized top-level schematic of the implemented design (with labels added), showing the neuron core, the six block-RAM cores, and the output ports x_out [15:0] and x_valid.

V. Experimental Protocol

The NASA PCoE dataset [16] cycles 18650 cells at room temperature. Eight discharge cycles spanning the life of each of B0005, B0006, B0007, and B0018 were used, over which SOH falls from 1.000 to 0.714, 0.583, 0.758, and 0.723. Each cycle was resampled to 64 points over its full discharge (32 cycles, 2048 samples), min–max normalized with training-cell statistics only, quantized to Q6.10, and fed to the reservoir, reset at each cycle start. SOC labels come from trapezoidal Coulomb counting against each cycle's measured capacity, and SOH labels are capacity divided by the cell's first-cycle capacity.

In nested LOCO cross-validation, each cell is held out once; an inner LOCO loop over the other three selects ridge: $\lambda \in \{10^{-5}, \ldots, 10^{3}\}$, PCA dimension, and (for SOH) one of three formulations by minimum RMSE. The held-out cell is never used for normalization, selection, or fitting. The baseline is a one-RC The´venin EKF refitted in every fold: fifth-order polynomial pseudo-OCV, least-squares R_0 (9.0–10.6 mΩ), fixed R_1 = 10 mΩ and C_1 = 3000 F (the records lack relaxation segments), and the true initial SOC – an advantage the ESN does not get. It is a simple, reproducible baseline, not the best achievable observer. Metrics are MAE, RMSE, R^2, and maximum absolute error; pooled metrics concatenate all four held-out cells.

VI. Results

A. Bit-Exact Verification

Both runs match at all five stages: 6400/6400 updates in Run A and 200/200 in Run B, with no missing, extra, or undefined records. Fig. 4 shows the agreement between the RTL simulation between the RTL simulation and the golden model: Panels (a)-(d) compare all 6400 neuron updates at the MAC, biased-sum, tanh-input, and tanh-output stages (the zero-valued bias stage is verified by run B), and panel (e) plots four neuron trajectories over the full recurrent sequence, with the golden model as lines and the RTL as markers. The RTL reproduces the golden model exactly through the full recurrent sequence. Neuron updates are spaced 4280 ns apart at the 10 ns simulation clock, i.e., 42 807 cycles per update.

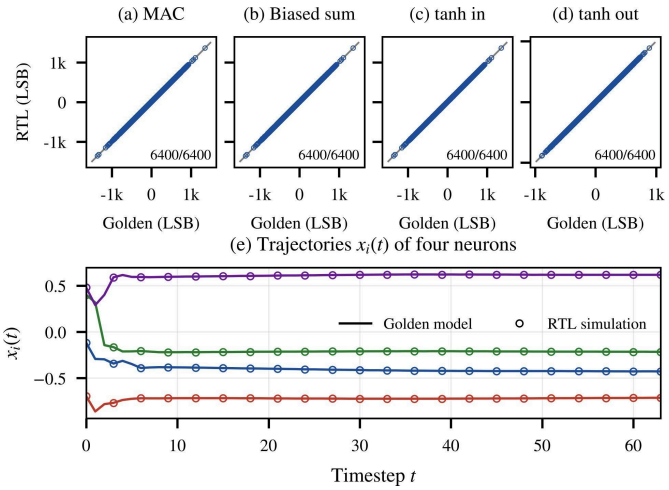


Fig. 4. Bit-exact agreement for Run A.

The SOC and SOH readouts were also verified in RTL simulation on the same 64-step sequence, and all 64 SOC outputs and the SOH output word matched the golden model bit for bit. The final SOH estimate is 0.7762 against a measured value of 0.7484, and the SOC at the final step is 0.0101.

B. Cross-Cell SOC Estimation

Table I states that, pooled over the four held-out cells, the bit-exact reservoir attains R^2 = 0.922 (RMSE 0.088) against 0.898 (RMSE 0.101) for the EKF, with 21% lower MAE. Fig. 5 shows the reference and estimated SOC over a mid-life cycle of each held-out cell. The advantage is not uniform: the ESN is clearly better on B0005 and B0007, marginally worse on B0006, and worse on B0018. The largest error (0.50) is the final sample of one B0007 cycle, where voltage collapses to 1.81 V, below the training range, and the linear readout extrapolates.

TABLE I
SOC Estimation on Held-Out Cells (Nested LOCO)

Held-out cell	Method	MAE	RMSE	R^2	Max. err.
B0005	ESN	**0.038**	**0.048**	**0.978**	**0.121**
	EKF	0.090	0.108	0.885	0.224
B0006	ESN	0.081	0.101	0.902	**0.211**
	EKF	**0.077**	**0.095**	**0.913**	0.275
B0007	ESN	**0.041**	**0.054**	**0.969**	0.502
	EKF	0.088	0.103	0.888	**0.187**
B0018	ESN	0.109	0.126	0.841	0.268
	EKF	**0.084**	**0.099**	**0.903**	**0.174**
Pooled	ESN	**0.067**	**0.088**	**0.922**	0.502
	EKF	0.085	0.101	0.898	**0.275**

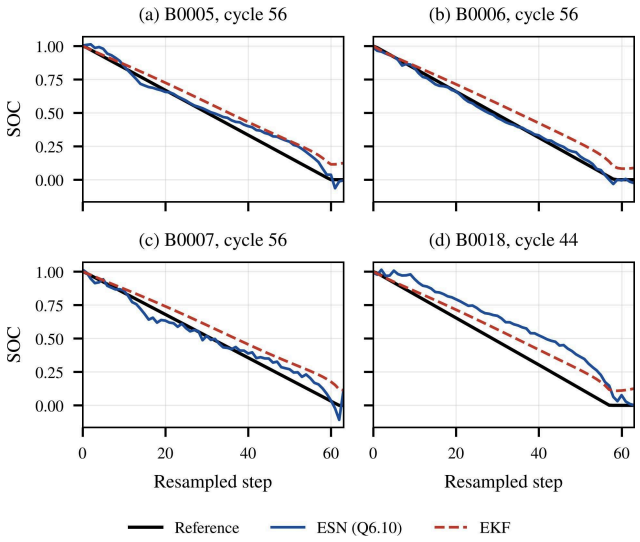


Fig. 5. Reference and estimated SOC over a mid-life cycle of each cell when it is held out.

C. Ablation and Seed Variability

Rerunning the identical nested protocol on two other representations shows where the gain comes from (Table II). A ridge readout on the four raw inputs, with no reservoir, reaches only $R^2 = 0.875$; an unquantized floating-point copy of the deployed reservoir reaches 0.921, versus 0.922 in Q6.10. The reservoir drives the gain, and quantization costs no measurable accuracy. As the deployed reservoir is one random draw, ten further reservoirs were drawn from the same recipe: pooled SOC R^2 was 0.923 ± 0.005 (range 0.916–0.933), above the EKF for every seed, and B0018 and B0006 were the two hardest cells in every draw. The SOH seed sensitivity was not measured.

TABLE II
POOLED SOC ACCURACY: RESERVOIR AND QUANTIZATION ABLATION

Representation	MAE	RMSE	R^2
Raw inputs, no reservoir	0.093	0.112	0.875
Floating-point reservoir (same weights)	0.068	0.089	0.921
Q6.10 reservoir (deployed)	**0.067**	**0.088**	**0.922**

D. Cross-Cell SOH Estimation and Overfitting

Inner validation selected the cycle-mean formulation (3) with $k = 8$ in every fold. On every held-out cell the readout cuts RMSE by 47–64% relative to predicting the training-set mean, reaching a pooled $R^2 = 0.782$ (RMSE 0.054; Table III, Fig. 6); with only 8 cycles per cell, per-cell values are noisy.

TABLE III
PER-CYCLE SOH ESTIMATION ON HELD-OUT CELLS (NESTED LOCO)

Held-out cell	MAE	RMSE	R^2	Max. err.	Mean-pred. RMSE
B0005	0.042	0.055	0.743	0.121	0.111
B0006	0.054	0.060	0.807	0.080	0.165
B0007	0.053	0.056	0.610	0.082	0.107
B0018	0.033	0.044	0.765	0.106	0.091
Pooled	**0.046**	**0.054**	**0.782**	**0.121**	**0.121**

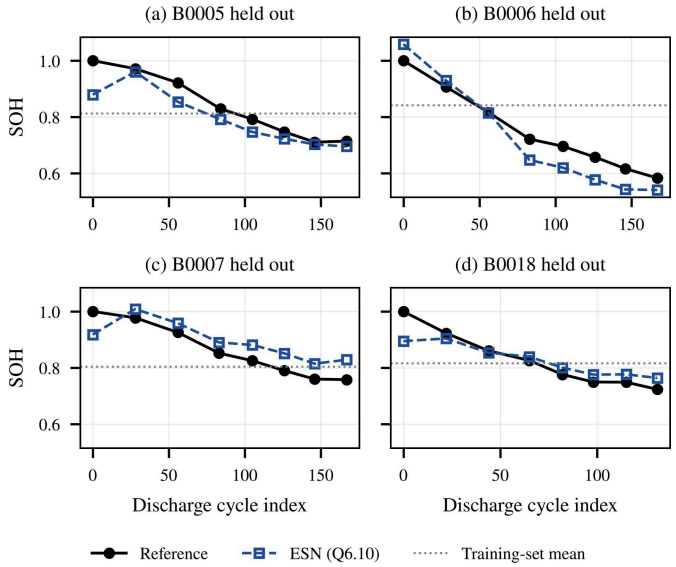


Fig. 6. Reference and estimated per-cycle SOH for each held-out cell; the dotted line is the training-set mean.

An earlier protocol that trained on B0005 alone overfitted badly: its SOH R^2 on the other three cells was 0.388 (0.650 in training), and single-cell readouts reached R^2 as low as −137 on other cells, because one cell gives only eight SOH labels for a 100-dimensional regression. Cross-cell training (24 cycles), cycle-level prediction, and an eight-component projection raise SOH R^2 on those same three cells to 0.791 with no negative fold, while SOC changes little.

E. Synthesis, Power, and Timing

Because synthesis removes logic that drives no output, the new state $x_i(t + 1)$, its write strobe, and the MAC saturation flag are brought out to top-level ports, so the complete datapath is retained. Vivado 2023.1 post-route results on an XC7A100T (Table IV) give 318 LUTs, 311 registers, 11.5 BRAM tiles, and two DSP slices; per module, the neuron core uses 259 LUTs and both DSPs (MAC: 168 LUTs), the tanh table four RAMB36 blocks, and the recurrent-weight memory five. Timing closes at 80 MHz with 0.023 ns setup slack; the critical path is the single-cycle MAC (weight read, DSP multiply, 40-bit rounding carry chain, saturation). One update takes 42 807 cycles, reflecting the fully serial single-MAC schedule; the resulting 535.1 μs latency still leaves about 18.7× headroom in a 10 ms BMS period. Power is Vivado's vectorless estimate (confidence: medium), about 81% static, not an activity-driven or silicon measurement. Fig. 7 shows the Digilent Arty A7-100T board (XC7A100T) that was used.

TABLE IV
POST-ROUTE RESOURCES AND PER-UPDATE TIMING/POWER (ARTIX-7 XC7A100T)

Clock	Total power (W)	Dynamic power (W)	WNS (ns)	Latency (µs)	Energy[a] (µJ)
80 MHz	0.120	0.023	+0.023	535.1	64.2
Resources: 318 LUTs (0.50%), 311 FFs (0.25%), 11.5 BRAM (8.5%), 2 DSP48E1 (0.83%)					

[a]Total on-chip power × update latency.

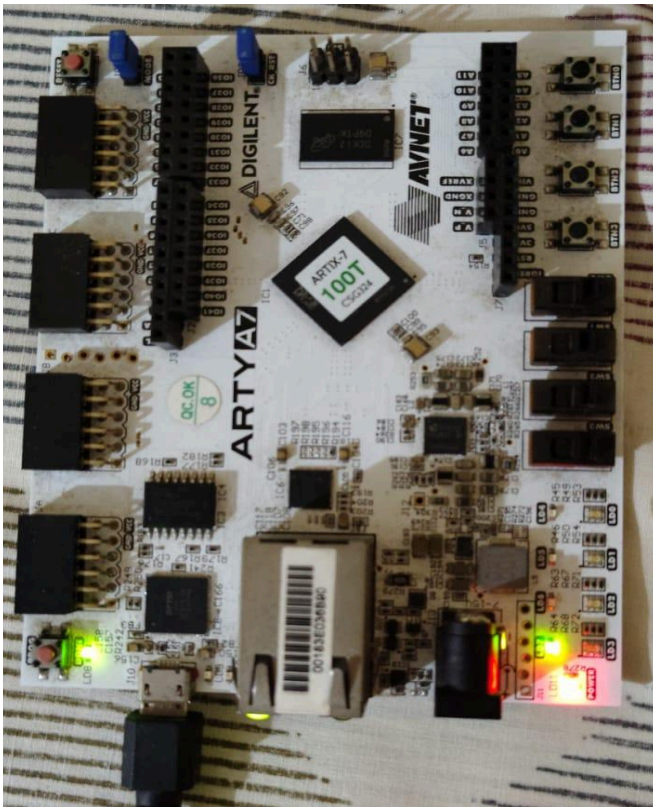


Fig. 7. Digilent Arty A7-100T development board (XC7A100T) running the Q6.10 ESN design.

VII. Discussion and Limitations

The recurrent weights never change, so the hardware needs no training logic or weight-update path – only one MAC core, read-only memories, and a small table – and, because readouts are trained on bit-exact states, the reported accuracies are those of the quantized hardware itself. The EKF is deliberately simple; a two-RC EKF with identified dynamics would likely do better, so the result reads as "competitive with a reasonable EKF at small hardware cost."

Limitations: (i) the serial schedule and the 80 MHz single-cycle MAC could be accelerated by pipelining, parallel lanes, or skipping the zeros of the 90%-sparse $\mathbf{W}$; (ii) resampling each discharge to 64 points over its own full length implicitly encodes its duration, and the cycle-mean SOH feature needs the complete discharge, so results are those of an offline full-sequence evaluation, not yet a causal online estimator; and (iii) power is a vectorless estimate, and running the 80 MHz core from the board's 100 MHz oscillator requires an on-chip clock synthesizer (MMCM).

VIII. Conclusion

A 100-neuron Q6.10 ESN reservoir for battery SOC/SOH estimation was implemented in Verilog for an Artix-7 FPGA and verified bit-exact against an independent golden model at five pipeline stages over a complete real discharge sequence, using 318 LUTs, 311 registers, 11.5 BRAM tiles, and two DSP slices at 80 MHz (0.120 W estimated, 64.2 µJ per update). Under nested leave-one-cell-out (LOCO) validation on NASA PCoE cells, the resulting estimator reached a pooled SOC R^2 of 0.922 against 0.898 for a refitted EKF (R^2 is the share of true SOC variance the estimate explains, i.e., 92.2% against 89.8%), a margin driven by the reservoir rather than the number format and stable across ten reservoir draws (0.923 ± 0.005), and a per-cycle SOH R^2 of 0.782 after cross-cell training removed severe single-cell overfitting. Fixed-interval preprocessing and dynamic load profiles are the next steps.

Data Availability

The NASA PCoE dataset used in this work is publicly available [16]. Details of the hardware design and evaluation procedure are given in Sections IV and V.

References

[1] M. A. Hannan, M. S. H. Lipu, A. Hussain, and A. Mohamed, "A review of lithium-ion battery state of charge estimation and management system in electric vehicle applications: Challenges and recommendations," *Renew. Sustain. Energy Rev.*, vol. 78, pp. 834–854, 2017.

[2] G. L. Plett, "Extended Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 3. State and parameter estimation," *J. Power Sources*, vol. 134, no. 2, pp. 277–292, 2004.

[3] P. Li, H. Wang, Z. Xing, K. Ye, and Q. Li, "Joint estimation of SOC and SOH for lithium-ion batteries based on EKF multiple time scales," *J. Intell. Manuf. Spec. Equip.*, vol. 1, no. 1, pp. 107–120, 2020.

[4] E. Chemali, P. J. Kollmeyer, M. Preindl, R. Ahmed, and A. Emadi, "Long short-term memory networks for accurate state-of-charge estimation of Li-ion batteries," *IEEE Trans. Ind. Electron.*, vol. 65, no. 8, pp. 6730–6739, 2018.

[5] H. Jaeger, "The 'echo state' approach to analysing and training recurrent neural networks," GMD—German Nat. Res. Center Inf. Technol., Bonn, Germany, GMD Rep. 148, 2001.

[6] W. Maass, T. Natschläger, and H. Markram, "Real-time computing without stable states: A new framework for neural computation based on perturbations," *Neural Comput.*, vol. 14, no. 11, pp. 2531–2560, 2002.

[7] M. Lukoševičius and H. Jaeger, "Reservoir computing approaches to recurrent neural network training," *Comput. Sci. Rev.*, vol. 3, no. 3, pp. 127–149, 2009.

[8] H. Jaeger and H. Haas, "Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication," *Science*, vol. 304, no. 5667, pp. 78–80, 2004.

[9] G. L. Plett, "Sigma-point Kalman filtering for battery management systems of LiPB-based HEV battery packs: Part 2. Simultaneous state and parameter estimation," *J. Power Sources*, vol. 161, no. 2, pp. 1369–1384, 2006.

[10] M. Yilmaz, E. Cinar, and A. Yazici, "A transformer-based model for state of charge estimation of electric vehicle batteries," *IEEE Access*, vol. 13, pp. 33035–33048, 2025.

[11] G. Tanaka *et al.*, "Recent advances in physical reservoir computing: A review," *Neural Netw.*, vol. 115, pp. 100–123, 2019.

[12] M. R. Kamarudin, M. S. Mispan, M. N. S. Zainudin, and H. Sofian, "Reservoir spiking neural networks for accurate state-of-charge estimation in battery management systems," *Turkish J. Eng.*, vol. 10, no. 2, pp. 407–417, 2026.

[13] V. M. Gan, "A cost-efficient digital ESN architecture on FPGA," M.S. thesis, Virginia Tech, Blacksburg, VA, USA, 2020.

[14] E. Rapuano, T. Pacini, and L. Fanucci, "A post-training quantization method for the design of fixed-point-based FPGA/ASIC hardware accelerators for LSTM/GRU algorithms," *Comput. Intell. Neurosci.*, 2022.

[15] S. Yun, J. Jeon, E. Lee, T. Jeong, and S. Kim, "FPGA implementation of battery state-of-charge estimation using extended Kalman filter and dynamic sampling," *World Electr. Veh. J.*, vol. 16, no. 10, art. 587, 2025.

[16] B. Saha and K. Goebel, "Battery data set," NASA Ames Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA, USA, 2007.